



JOR-V3.1 Collision Risk Module

STANDARD OPERATING PROCEDURE & USER MANUAL

This document provides comprehensive, step-by-step instructions for operating the **Collision Risk Index (CRI)** module within the James Orion Report (JOR) Bayesian Fusion Framework. This localized ecosystem maps physical sensor telemetry and anomalous flight behavior into a quantified Aviation Hazard Score.

1. Environment & Architecture Overview

The Collision Risk Module operates within a completely self-contained folder structure to prevent analytical cross-contamination. Ensure your local environment contains the following system assets before starting execution:

jor_fusion.py — Interactive terminal scoring interface and baseline configuration. Holds global calibration constants.

jor_pymc.py — Highly optimized, vectorized PyMC sampling engine handling evidentiary probability distributions.

jor_pymc_runner.py — The bridge automation script that runs the MCMC engine, calculates risk multipliers, and performs the safety audit.

app.py — Frontend Streamlit application rendering the operational Air Safety Command Center.

jor_scores.csv — *Automatically generated by the system backend after the first tracking profile is saved.*

2. Step-by-Step Operational Workflow

Follow this exact chronological loop to ingest a tracking file, generate probabilistic model metrics, and deploy dashboard overlays.

Step 1: Initial Case Ingestion & Scoring

To capture human witness profiles and sensor telemetry configurations, execute the standalone scoring CLI interface.

1. Open a terminal process and step into the target module workspace:

```
cd Collision_Risk
```

2. Initialize the scoring matrix tool:

```
python jor_fusion.py
```

3. **Configuration Verification:** The terminal will render a snapshot of current file parameters (`PRIOR_NH` , weights). Review and verify compliance.
4. **Data Input Matrix:** Progress through sequential rubric menus to assign scores for:
 - Witness Credibility (C)
 - Environmental / Observation Conditions (E)
 - Physical / Sensor Evidence (P)
 - Flight Behavior Classification (None, Minor, Moderate, or Major Anomaly)
5. Input `y` when prompted to finalize and save the data tracking session.

 **AUTOMATED INITIALIZATION NOTICE:** If `jor_scores.csv` is completely missing on your system, the ledger is generated automatically upon closing the scoring script. It injects structured headers and ensures trailing float values are cleaned to prevent floating-point tail leakage across rows.

Step 2: Execute MCMC Processing Engine & Risk Audit

Transform raw categorical scoring values into rigorous, sampled posterior parameter boundaries using the processing automation script.

1. Execute the operational script bridge directly from your terminal session:

```
python jor_pymc_runner.py
```

2. The sampler initializes 4 parallel Markov Chain Monte Carlo (MCMC) tracking chains to resolve evidentiary distributions, compiling 1,000 baseline tracking draws per chain.
3. **Automated Index Mapping:** The bridge script pulls the probabilistic mean output (SOP_{Mean}) and executes the core safety risk calculation:

$$CRI = SOP_{Mean} \times (1 + FlightMod)$$

1. The system executes a discrete classification sweep, indexing tracks automatically into designated **Low**, **Elevated**, or **Critical** hazard threat tiers. Final records are written smoothly to disk.

Step 3: Launch the Safety Command Center Display

Render interactive visualizations and telemetry charts for flight safety officers or tactical intelligence analysis briefings.

1. Launch the localized user interface instance:

```
streamlit run app.py
```

2. The interface generates a secure local server access point (default: `http://localhost:8501`). Move to your browser to interact with data assets.
3. **Fallback Simulation Engine:** If running a fresh instance without historical telemetry logs, the dashboard automatically deploys a mock dataset (e.g., *USS Nimitz CSG*) to ensure operational utility from zero-state initialization.
4. Use the sidebar control " **Hot-Reload Core Telemetry**" to manually flush working caches and instantly pull live modifications from analytical pipeline iterations.

3. Core Analytical Frameworks

The layout features three segregated analytical viewports designed specifically for mitigating threat vectors inside the active operations environment:

1. Kinetic Hazard Frontier

Maps spatial tracking physics by indexing the probabilistic SOP_{Mean} against the mathematical kinematic scalar multiplier (`Flight_Mod`). This viewport relies on layered, background logistic contour gradients to classify emerging flight risk thresholds. Scatter node dimensions dynamically expand relative to the statistical uncertainty range (**95%** Confidence Intervals) calculated by the backend MCMC sampling matrix.

2. Threat Intelligence Quadrant Map

A coordinate plotting tool aligning base physicality confidence levels against absolute calculated CRI safety scores. This grid maps physical assets from ambiguous environmental artifacts. It features a visible, non-negotiable hazard indicator marking the **Critical Action Limit (0.75)** boundary line to guide active escalation procedures.

3. Evidentiary Uncertainty Forest

An error-bar plot highlighting the precise dispersion limits between $CI_{2.5\%}$ and $CI_{97.5\%}$ across Non-Human Hypothesis Posterior distributions. This enables tracking personnel to evaluate if a critical warning indicator is triggered by substantial physical target profiles or massive variance tracking artifacts.

4. Managing Calibration & Sensitivity Analyses

To calibrate uncertainty tolerances or adjust weights to align with changing intelligence validation baselines:

 **CRITICAL CONFIGURATION PROTOCOL:** Global constants controlling underlying weights and limits are managed strictly via code adjustments at the head section of `jor_fusion.py`.

Option 1: Reproducible Calibration and MCMC Sensitivity Sweeps (Recommended)

1. Open `jor_fusion.py` using an external configuration or code editor.
2. Edit the designated parameters located below the explicit `# Configuration` header boundaries:

```
PRIOR_NH = 0.20      # Target prior probability
CALIBRATION_K = 0.20 # Calibration coefficient scalar
WEIGHT_C = 0.40      # Witness weight modifier
WEIGHT_E = 0.30      # Environmental conditions weight modifier
WEIGHT_P = 0.30      # Physical evidence weight modifier
```

3. Commit structural updates to file disk.
4. Run `python jor_pymc_runner.py` inside a **completely clean, fresh process environment**. The automation bridge loads updated variables directly from code structure, forcing the MCMC sampler to track modifications perfectly.
5. *Ensure baseline defaults are restored manually once sensitivity verification sweeps conclude.*

Option 2: Memory-Transient Standalone Diagnostics

1. Input `y` at the terminal challenge prompt to alter constants interactively during runtime inside `python jor_fusion.py`.
2. *Operational Risk:* Variable updates exist exclusively within the context memory buffer of that temporary shell environment. Changes **will not pass through** when executing `jor_pymc_runner.py`, as the sampler script reads fresh parameters from file arrays during startup.